Preprocessing.ipynb – Technical Report

Data Preprocessing Pipeline Review — Loan Default Prediction (XAI) Project

---

1. Purpose of This File in the Project

Preprocessing.ipynb is the **second stage** of the machine learning pipeline, sitting directly between the exploratory analysis (EDA.ipynb) and model training. Its job is to turn the raw, human-readable dataset into a clean, encoded, scaled, and balanced numeric dataset that a model such as LightGBM/XGBoost can actually learn from. Concretely, it is responsible for:

- Cleaning column names and dropping non-predictive identifiers.
- Encoding binary and categorical variables into numeric form.
- Detecting and handling missing values, duplicates, and outliers.
- Reducing skewness in continuous variables via log transformation.
- Engineering new, higher-signal composite features (risk scores, ratios, interaction terms).
- Splitting the data into train / validation / test sets without leakage.
- Correcting class imbalance (SMOTETomek) on the training set only.
- Scaling features and persisting the fitted scaler and feature list for deployment.

Compared to EDA.ipynb, this notebook is considerably more mature and complete: it is well documented, uses correct data-leakage-avoidance practices (e.g. fitting the scaler and SMOTETomek only on the training split), and persists reusable artifacts for the backend API.

2. Declared Dataset Schema (Notebook Introduction)

The notebook opens with a documentation table describing the raw dataset (Kaggle – nikhil1e9/loan-default) and each column's intended treatment:

Column	Type	Declared Treatment
LoanID	str	Unique identifier – dropped
Age	int	Borrower age
Income	float	Right-skewed → log transform (as declared)
LoanAmount	float	Right-skewed → log transform (as declared)
CreditScore	int	Declared as strongest predictor (corr ≈ –0.20)
MonthsEmployed	int	Months at current employer
NumCreditLines	int	Number of open credit lines
InterestRate	float	Loan interest rate %

LoanTerm	int	Loan duration in months
DTIRatio	float	Declared as 2nd strongest predictor
Education	str	Ordinal (High School → PhD)
EmploymentType	str	One-hot encoded
MaritalStatus	str	One-hot encoded
HasMortgage / HasDependents / HasCoSigner	str	Yes/No → 1/0
LoanPurpose	str	One-hot encoded
Default (target)	int	1 = defaulted (≈11.6%), 0 = not (≈88.4%)

The notebook also states the class imbalance ratio up front as 7.7:1, to be addressed with SMOTETomek and class weighting — see Section 4 for the actual measured ratio.

3. Step-by-Step Walkthrough of the Pipeline

1 · Libraries

Imports pandas, numpy, joblib, matplotlib/seaborn, and scikit-learn utilities (train_test_split, StandardScaler, IsolationForest, LocalOutlierFactor) plus imbalanced-learn (SMOTE, SMOTETomek). Warnings are suppressed and display options set.

2 · Load Raw Data

Changes the working directory up one level and loads `data/raw/Loan_default.csv` — 255,347 rows × 18 columns, matching the EDA notebook's figures exactly.

3 · Class Imbalance Check

Plots a count chart and pie chart of the target variable and computes the imbalance ratio explicitly before any other processing — correctly treated as the first, most important, decision driver.

4 · Drop ID & Clean Column Names

Drops LoanID (no predictive value) and standardizes all column names to lower-case, underscore-separated form (e.g. CreditScore → creditscore).

5 · Data Types & Binary Encoding

Runs df.info() then maps the three Yes/No columns (hasmortgage, hasdependents, hascosigner) to 1/0.

6 · Missing Values

Computes missing count/percentage per column. Includes a fallback branch that would median-impute numeric columns and mode-impute categorical columns if any were found.

7 · Remove Duplicates

Drops exact duplicate rows and resets the index.

8 · Outlier Detection (Isolation Forest ∩ LOF)

Applies two independent outlier detectors — Isolation Forest and Local Outlier Factor, each with 5% contamination — on the numeric columns, and only flags a row as an outlier if **both** methods agree (intersection). This is a deliberately conservative, high-confidence approach. Rows are then either dropped (if below the 5% threshold) or clipped to the IQR bounds (if above it).

9 · Skewness Analysis & Log Transformation

Computes skewness for the 9 core numeric features and applies log1p to Income and LoanAmount, which the notebook's own documentation assumes have skewness ≈ 2.0. See Section 5 for a discrepancy found here.

10 · Encode Categorical Features

Education is ordinal-encoded (High School=0 → PhD=3, since a natural order exists); EmploymentType, MaritalStatus, and LoanPurpose are one-hot encoded with `drop_first=True` (no natural order between categories).

11 · Feature Engineering

Introduces 5 original interaction features (loan_to_income, monthly_income, employment_ratio, creditscore_band, high_risk_flag) plus 7 new composite features (monthly_payment_est, payment_to_income, debt_burden_score, a weighted risk_score, log_loan_to_income, is_very_high_risk, age_employment_interaction). The stated philosophy is that combinations of risk factors (e.g. high DTI + low credit score + high interest rate) carry more signal than any single raw feature.

11.1 · Feature Correlation with Target (validation)

Computes the absolute Pearson correlation of every numeric feature with the target and plots the top 20, using 0.05 as a significance threshold. This step is meant to validate that the engineered features actually carry predictive signal — see Section 5 for what the results actually show.

12 · Train / Validation / Test Split

A standard 70/15/15 stratified split (first 70/30, then the 30% temp split 50/50), explicitly preserving class balance across all three sets, with the test set reserved for a single, final evaluation.

13 · Handle Class Imbalance — SMOTETomek

Applies SMOTETomek (SMOTE oversampling of the minority class combined with Tomek-link removal of noisy majority samples) — correctly applied to the training set only, leaving validation and test untouched to avoid data leakage.

14 · Scaling

Fits a StandardScaler on the (resampled) training data only, then transforms validation and test with the same fitted scaler — another correct anti-leakage practice. The scaler is later persisted for reuse at inference time.

15 · Save Processed Data & Artifacts

Saves the full processed dataset and the scaled train/val/test splits as Parquet files under `data/processed/`, and persists `models/scaler.pkl` and `models/feature_names.pkl` via joblib for use in the backend inference service.

16 · Preprocessing Summary

Closes with a markdown table recapping every step and its rationale, and a stated key insight: raw features have weak correlation with Default (max ≈ –0.20 for CreditScore), so the engineered composite features are expected to carry most of the predictive signal for tree-based models.

4. Key Findings (From Actual Executed Output)

Unlike EDA.ipynb, this notebook's saved outputs show it ran successfully end-to-end. The measured results were extracted directly from the saved cell outputs:

Stage	Result
Raw shape	255,347 rows × 18 columns
Class imbalance (measured)	7.6 : 1 — Default = 11.61% of rows (29,653 rows)
Missing values	None found
Duplicate rows removed	0
Outliers found (Isolation Forest ∩ LOF)	872 (below the 5% / 12,767-row threshold) → dropped
Shape after outlier removal	254,475 rows × 17 columns
Shape after encoding	254,475 rows × 25 columns
Shape after feature engineering	254,475 rows × 37 columns
Final feature count (X, pre-split)	36 features
Train / Val / Test split	178,132 / 38,171 / 38,172 rows — 11.6% default rate preserved in all three
Train set after SMOTETomek	304,952 rows, perfectly balanced (152,476 / 152,476)
Scaling check	Train mean ≈ 0.0000, train std ≈ 1.0000 (correct)
Artifacts saved	scaler.pkl, feature_names.pkl (36 features), 6 Parquet files

5. Issues and Discrepancies Found

This notebook is well engineered and mostly free of the structural bugs found in EDA.ipynb (no undefined variables, no broken cells). However, comparing its own written assumptions against its actual measured output surfaces two notable discrepancies worth flagging:

5.1 Log transformation assumption does not match measured skewness

Section 9's markdown states: "Income and LoanAmount have skewness ~2.0 — log1p normalises the distribution." The actual computed skewness, taken directly from the executed output, shows both features are already essentially symmetric before any transform:

```
income skew: -0.00 | loanamount skew: -0.00 (before log1p)
```

After applying log1p anyway, the skewness moves *away* from zero rather than toward it:

```
income skew: -0.00 → -0.78 | loanamount skew: -0.00 → -1.25 (after log1p)
```

In other words, on this particular run of the dataset the log transform introduces negative skew rather than correcting positive skew as intended. This suggests either the documentation comment was carried over from an earlier version of the dataset/notebook, or the skewness check should be re-run and used to conditionally decide whether to apply the transform, rather than applying it unconditionally to a fixed column list.

5.2 CreditScore and DTIRatio do not appear among the top correlated features

The notebook's introduction (Section 1) explicitly describes CreditScore as "the strongest predictor, corr=–0.20" and DTIRatio as the "2nd strongest predictor." However, the actual correlation-with-target output computed later in the same notebook (Section 11.1) lists the top features above the 0.05 threshold as follows — and neither creditscore nor dtiratio appears in that list at all:

```
loan_to_income 0.178 | payment_to_income 0.170 | age 0.168 | age_employment_interaction 0.151 |
interestrate 0.131 ...
```

This is a meaningful mismatch between the documented assumption used to justify the ordinal/log-transform design decisions and what the data in this run actually shows. It is worth double-checking whether this is expected (e.g. CreditScore's relationship with Default may be non-linear and better captured by `creditscore_band`, which is not shown in the same list either), or whether it points to a difference between the dataset version described in the documentation and the one actually loaded at `data/raw/Loan_default.csv` during this run.

5.3 Good practices confirmed (no leakage)

On the positive side, three common preprocessing mistakes are correctly avoided:

- SMOTETomek is fit and applied only on the training split (validation/test distributions remain at the natural 11.6% default rate).
- StandardScaler is fit only on training data, then reused via transform() on validation and test.
- The train/validation/test split is stratified, so class balance is preserved identically (11.6%) across all three sets before resampling.

6. Recommendations

Re-validate the log-transform decision. Since income/loanamount skewness measured ≈0.00 on this run, either recompute skewness dynamically and apply log1p conditionally (only if |skew| exceeds a threshold, e.g. 0.75), or confirm whether the raw dataset changed since the ~2.0 skewness figure was documented.

Reconcile the CreditScore/DTIRatio correlation claim. Update the introductory table in Section 1, or add a short note in Section 11.1 explaining why these two features, despite being described as the strongest linear predictors, do not appear above the 0.05 correlation threshold in the current run (e.g. non-linear relationship better captured by creditscore_band / high_risk_flag / is_very_high_risk).

Surface creditscore_band and high_risk_flag correlations explicitly. Since these engineered features are meant to capture CreditScore's non-linear risk relationship, show their correlation with Default alongside the top-20 chart to confirm the substitution actually recovers the expected predictive signal.

Document the outlier-handling branch that wasn't exercised. This run took the "drop" branch (872 < 5% threshold). It would be useful to note in the notebook that the alternative "clip to IQR" branch exists for cases where outliers exceed the threshold, so future maintainers know both code paths are intentional.

Keep artifacts versioned. scaler.pkl and feature_names.pkl are correctly persisted for deployment; consider also saving the exact list of engineered feature formulas/version or a requirements/environment snapshot alongside them, so the backend inference pipeline stays in sync if the notebook changes in the future.

Overall assessment: Preprocessing.ipynb is a well-structured, leakage-aware pipeline that correctly separates fitting from transforming across train/validation/test, and produces reusable artifacts for deployment. The issues found are documentation/assumption mismatches rather than functional bugs, and are worth resolving mainly for reproducibility and reviewer confidence rather than because the pipeline is broken.

Report generated by reviewing Preprocessing.ipynb (35 cells) — code and previously saved cell outputs — for the depi_loan_default_xai project.